Note on 3-Coloring Problem*

Zhongkui Ma

February 19, 2022

1 Algorithm

1.1 Related Concepts

In the following discussion, the graph $G = (V(G), E(G))$ is an unweighted and undirected simple graph. Before explaining the algorithm, first, describe some related concepts.

Definition 1 *For the graph G, the union of the shortest path $\{S(r,i)\}$ obtained by the shortest path algorithm to all other vertices $i \in V$, taking a vertex $r \in V$ as the starting point, is $\cup_{i\in V} S(r,i)$. The* ***shortest-path tree*** *$T(r) = (V(T), E(T))$ is $\cup_{i\in V} S(r,i)$.*

Definition 2 *In the graph G, the shortest-path trees with vertices $r_1, r_2, r_3, \cdots$ as the root vertices are $T(r_1), T(r_2), T(r_3), \cdots$. And the* ***multi-point shortest-path forest*** *with $r_1, r_2, r_3, \cdots$ as the root vertices refers to $T(r_1, r_2, r_3, \cdots) = \cup_{j=1}^{n} s_{min}(j)$, where*

$$s_{min}(j) = \underset{s_{T(r_i)}}{min}\ s_{T(i)}(j) \tag{1}$$

is the shortest path of the shortest path of the vertex j in each shortest-path tree. For all edges e satisfying $e \in T(r_i)$ and $e \in T(r_1, r_2, r_3, \cdots)$, the subtree formed is called the subtree in the multipoint path forest $T'(r_i)$.

Each subtree $T'r_i$ in the multipoint shortest-path forest is disconnected because a vertex can only choose one shortest path. These subtrees are the parts of the shortest-path tree $\{T(r_i)\}$ that are closer to the root vertex r_i.

1.2 Description

Assuming there are three colors, color 1, 2, and 3. Colour 1 and 2 are specified as dark colors, and color 3 is specified as light colors. The parent and child vertices refer to the adjacenct vertices of one tree $T(r_i)$.

1. Select a vertex $r_0 \in G$ arbitrarily to join the root vertex set $\{r_i\}$.
2. Obtain the multi-point shortest-path forest $T(\{r_i\})$. Set the root vertices to light color, and all other vertices to dark color.
3. Traverse all vertices. $T(r_i)$ is one subtree of $T(\{r_i\})$. If the following conditions are meet, set i to light.
   - The current vertex $i \in T(\{r_i\})$ has one adjacent edge $(i,j) \in E(G) - E(T(\{r_i\}))$ and the distances $s_{T_i}(i)$ and $s_{T_j}(j)$ to the root vertices of their subtrees T_i and T_j are same. The edge (i,j) is called **horizontal edge**.
   - All these adjacent vertices $\{k|(i,k) \in E(G)\}$ of horizontal edges $\{(i,k)\}$ are all dark
4. Traverses and paint all vertices.

*This note does not guarantee the correctness and you can email to me (*zhongkui.ma@uqconnect.edu.au*) if you are interested or have problems with it.*

- Paint all light-colored vertices with color 3.
- Paint all dark-colored vertices alternately. If one dark-colored vertex i is not painted, use it as the starting point to traverse all adjacent dark-colored vertices by BFS, and paint one adjacent vertex dark color that is different from i if the distance to i is an odd number; paint the same color as i if the distance i is even.

5. Traverse all vertices. Check if there are two vertices with the same color.

   - If two vertices are not adjacent to the root vertex, select one of them as the root vertex; if two vertices are adjacent to multiple root vertices, select one vertex instead of a adjacent root vertex as the root vertex, and ensure that any two root vertices are not adjacent (after the first loop of the algorithm, the root vertex randomly selected at the beginning is removed). Each vertex can only be as root vertex once.
   - If there is no two vertices with the same color, three colors can color the graph G; otherwise, it does not meet the requirements.

   Repeat the above procedure 2–5 until a new root vertex cannot be selected or there are no two vertices with the same color.

1.3 Proof

Theorem 1 *In the shortest-path tree T, if a cycle contains a horizontal edge, it is an odd cycle. Therefore this circle cannot be colored by two colors (Figure 1).*

Proof *Because the distancees from the two vertices i and j of the horizontal edge (i, j) to the vertices of their subtree are the same, so if their two branches have an intersection, then the distance from each vertex to this intersection vertex is also the same. This circle contains this intersection; otherwise, the circle cannot exist. So the circle has an odd number of edges, a horizontal edge and two same number of sides from the two vertices to the intersection. So this circle has an odd number of vertices. Therefore this circle cannot be colored by two colors.* ■

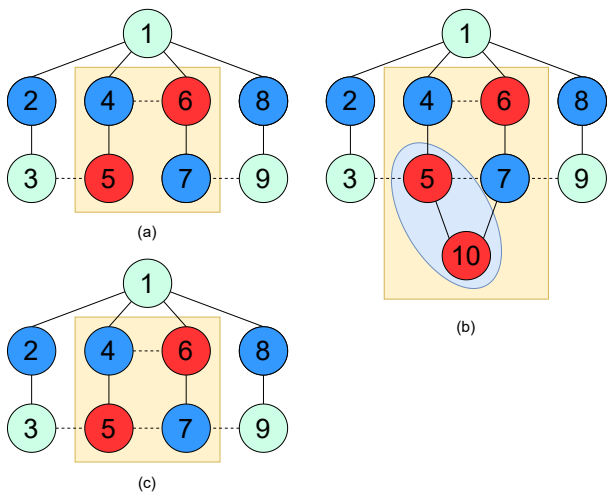

Figure 1: Examples of dark-colored circle (Figure (a) shows a 3-colorable graph without an dark odd circle. The yellow box of Figure (b) contains an dark odd circle with two adjacent vertices (in light blue box) of same color. The yellow box of Figure (c) contains a dark even circle.)

Theorem 2 *After step 2, any two adjacent vertices in the graph G cannot be light colors simultaneously.*

Proof *Because according to the above steps, if the adjacent vertices of the current vertex are not all dark, then it must remain the default dark.* ■

Theorem 3 *After step 3, if any two adjacent dark vertices in the graph G have the same color, they are in a dark odd circle R that does not contain the root vertex r.*

Proof *Because the root vertex r is light-colored, the dark circle R cannot contain it.*

If R is even circle, its vertices are painted with two different dark colors alternately according to the procedures; if R is odd circle, in any case, two adjacent dark vertices will appear with the same color. ■

Theorem 4 *The loop of algorithm will not fall into an infinite loop.*

Proof *Because each vertex can only be a root vertex once, so the algorithm will stop.* ■

Theorem 5 *After step 4, if there is a dark odd circle R in the graph G, then each vertices in the dark circle is connected to one of root vertices $\{r_i\}$.*

Proof *Because breadth-first traversal is used, when two dark vertices of R with the same color are adjacent to the root vertex after painting, then all the vertices of R are adjacent to some root vertices. And at the same time, any vertex in the circle R cannot be used as a new root vertex again, because these dark vertices have been root vertices once.*

When all the vertices $\{i\}$ of R are adjacent to one root vertex r (Figure 2), then for the graph $R' = R \cup r$ (R' contains R and r, as well as all edges between vertices of the circle R and r), there is no 3-coloring scheme. Because the odd circle R is 3-colorable, and the its vertices cannot have the same color with r.

When the vertices $\{i\}$ of R are adjacent to multiple root vertices (Figure 2), one of the root vertices will be replaced with a vertex in the dark circle. If the algorithm stop at this time, that means there is a special subgraph causing a loop that cannot split the existed dark loop (The Mycielski-Grötzsch graph that is triangle-free **?** *is an example).* ■

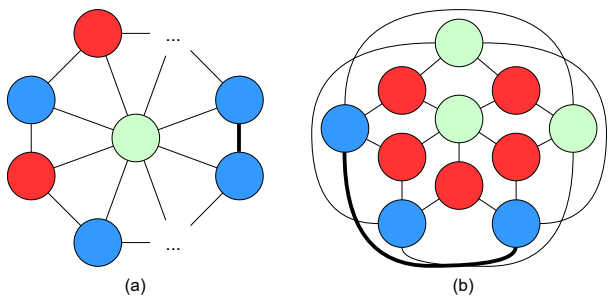

Figure 2: Examples of two sort of graph. (The thick edge shows two adjacent vertices with the same color and the green vertices are root vertices. Figure (a) shows a root vertex is adjacent to all the vertices of an odd circle. Figure (b) shows the Mycielski-Grötzsch graph **?** that causes a infinite loop of replacing root vertices without restricting once update of each vertex, and it is completely different from the situation of Figure (a).)

1.4 Complexity

Assuming the graph G has m edges and n vertices. The total time complexity if the algorithm is $O((m+n)n\log n)$. The complexity of Step 1 is $O(1)$. Step 2 is the time-complexity of the shortest-path algorithm. Step 3 needs calculating the distances of each vertices when traversing them, so the time complexity is $O(m+n)$. Step 4 and 5 both have complexity of $O(m+n)$. The total time complexity of this algorithm is $O((m+n)\log n)$.